PRACTICAL NO. 4

Raspberry Pi Based Oscilloscope

Aim

To replicate the signal visualization capability of an oscilloscope using Raspberry Pi and an analog-to-digital converter (ADS1115).

An oscilloscope is a laboratory instrument commonly used to display and analyze the waveform of electronic signals. In effect, the device draws a graph of the instantaneous signal voltage as a function of time. In this project the signal visualization of an oscilloscope is replicated using the Raspberry Pi and an analog-to-digital converter module. This requires the following steps:

- Perform analog-to-digital conversion of the input signal
- Prepare the resulting data for representation
- Plot the data on a live time graph

Analog Signals > Signal Processing > Visualize using the Graph

Hardware Requirements

- Raspberry Pi Model A/B/B+
- ADS1115 ADC
- Breadboard
- Jumper wires

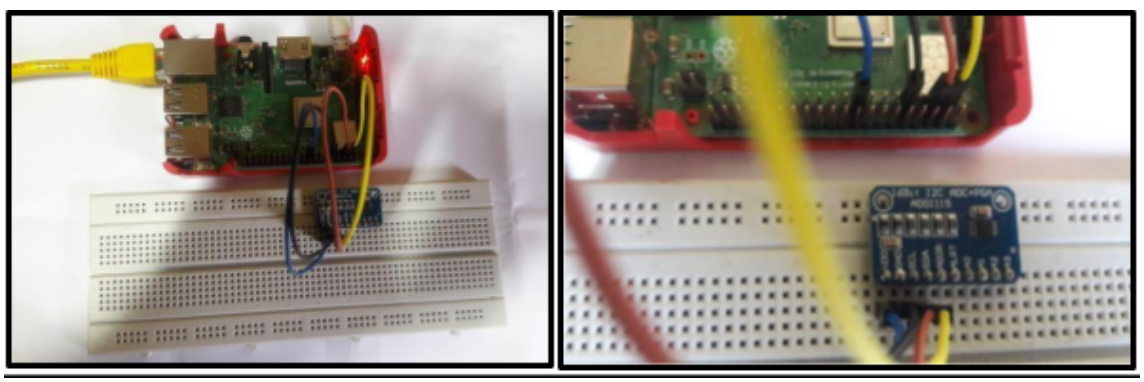
The **ADS1115** ADC chip converts analog input signals to digital values that the Raspberry Pi can visualize. It is needed because the Raspberry Pi does not have an on-board analog-to-digital converter.

Software Requirements

- Raspbian Stretch OS (or later)
- Adafruit module for interfacing with the ADS1115 ADC chip
- Python module **matplotlib** for data visualization
- Python module **drawnow** for live plot updates

Circuit Connections

ADS1115 ADC	Pin Number	GPIO Number
VDD	Pin 17	3.3V
GND	Pin 9	GND
SCL	Pin 5	GPIO 3
SDA	Pin 3	GPIO 2



ADS1115 connected to the Raspberry Pi through a breadboard

Step 1: Enable I2C

```
sudo raspi-config
```

When the configuration panel opens, select **Interface Options > I2C > Enable**.

Step 2: Update the Raspberry Pi

Update the system first so that everything on the OS is up to date and there are no compatibility issues with newly installed software.

```
sudo apt-get update
sudo apt-get upgrade
```

Step 3: Install the Adafruit ADS1115 library

Make sure you are in the home directory, then install the build essentials:

```
cd ~
sudo apt-get install build-essential python3-dev python3-smbus git
```

Clone the Adafruit library, change into its folder and run the setup file:

```
git clone https://github.com/adafruit/Adafruit_Python_ADS1x15.git
cd Adafruit_Python_ADS1x15
sudo python3 setup.py install
```

Step 4: Test the library and I2C communication

Before proceeding, test that the ADC communicates with the Raspberry Pi over I2C using an example script that comes with the library. While in the **Adafruit_Python_ADS1x15** folder, change to the examples directory and run **simpletest.py**, which displays the values of the four ADC channels in a table:

```
cd examples
python3 simpletest.py
```

Alternatively, the following CircuitPython-based test script reads channel 0 (install it with **pip3 install adafruit-circuitpython-ads1x15**):

Code: test script

```
import time
import board
import busio
import adafruit_ads1x15.ads1115 as ADS
from adafruit_ads1x15.analog_in import AnalogIn

# Create the I2C bus
i2c = busio.I2C(board.SCL, board.SDA)

# Create the ADC object using the I2C bus
ads = ADS.ADS1115(i2c)

# Create single-ended input on channel 0
chan = AnalogIn(ads, ADS.P0)

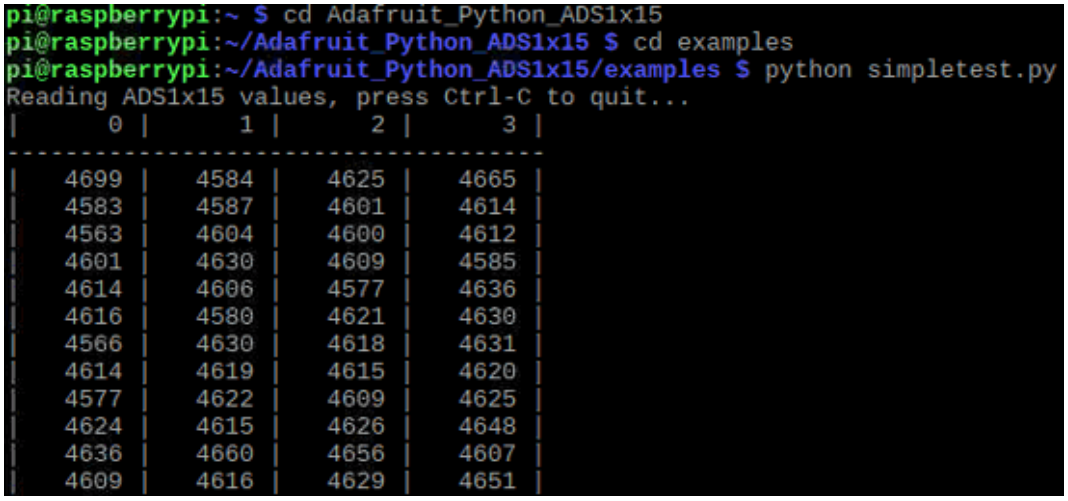
# Create differential input between channel 0 and 1
# chan = AnalogIn(ads, ADS.P0, ADS.P1)

print("{:>5}\t{:>5}".format("raw", "v"))

while True:
    print("{:>5}\t{:>5.3f}".format(chan.value, chan.voltage))
    time.sleep(0.5)
```

Output

If the I2C module is enabled and the connections are good, you should see the channel data as shown below. If an error occurs, check that the ADC is properly connected to the Pi and that I2C is enabled.

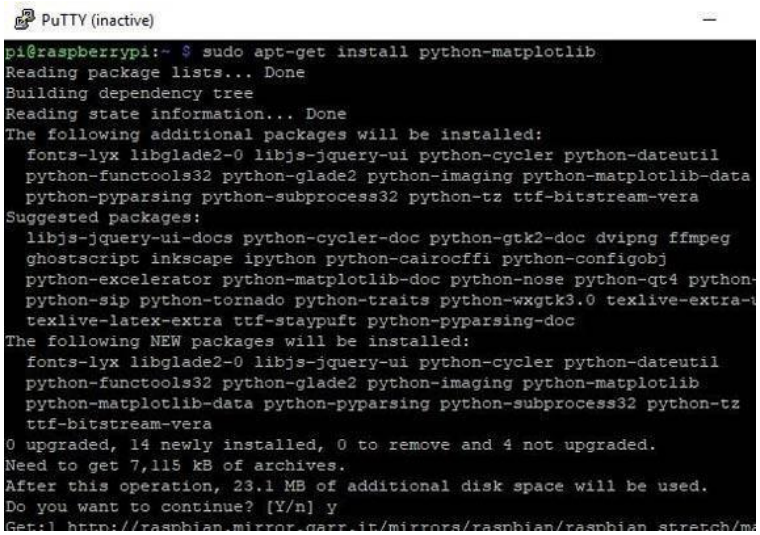


Sample output of simpletest.py (ADC channel values)

Step 5: Install matplotlib

The matplotlib module is used to plot graphs in Python:

```
sudo apt-get install python3-matplotlib
```



Installing matplotlib

Step 6: Install the drawnow Python module

The **drawnow** module provides live updates to the data plot. It is installed with pip, so make sure pip is installed first:

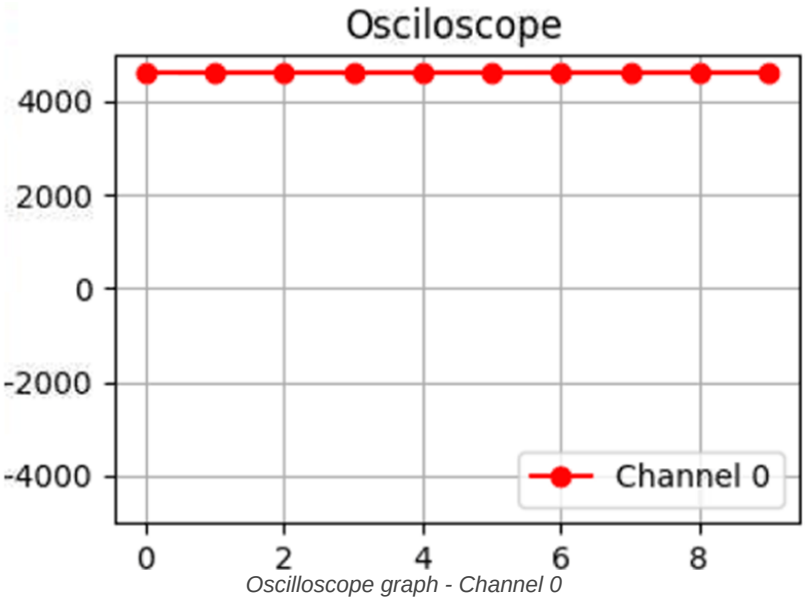
```
sudo apt-get install python3-pip
sudo pip3 install drawnow
```

Step 7: Download and run the graph code

```
git clone https://github.com/tyitraspberrypi/Adafruit_Python_ADS1x1z_graph.git
```

Output

The live oscilloscope graph of Channel 0 is plotted as shown below.



Oscilloscope graph - Channel 0

PRACTICAL NO. 7

Interfacing a 16x2 LCD (I2C) with Raspberry Pi

Aim

Interface a 16x2 LCD module with Raspberry Pi to display user-defined messages on the screen.

Requirements

- Raspberry Pi 400
- LCD 1602 with I2C module (PCF8574 backpack)
- 4 female-to-female jumper wires

Circuit Connections (I2C LCD)

LCD Pin	Connect to Raspberry Pi GPIO
GND	Pin 6 (GND)
VCC	Pin 4 (5V)
SDA	Pin 3 (GPIO 2 - SDA)
SCL	Pin 5 (GPIO 3 - SCL)

Connection diagram:

```
LCD       Pi 400 GPIO
----      ------------
GND   --> GND (Pin 6)
VCC   --> 5V  (Pin 4)
SDA   --> SDA (GPIO 2 / Pin 3)
SCL   --> SCL (GPIO 3 / Pin 5)
```

Step 1: Enable I2C on the Raspberry Pi

```
sudo raspi-config
```

Go to **Interface Options > I2C > Enable**, then reboot the Pi.

Step 2: Install the required packages

```
sudo apt update
sudo apt install i2c-tools python3-smbus python3-pip
pip3 install RPLCD
```

On Raspberry Pi OS Bookworm, pip may require the extra option **--break-system-packages** (or use a virtual environment).

Check that the LCD is detected:

```
i2cdetect -y 1
```

Sample output:

```
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- 27 -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

Note down the address shown (usually **0x27** or **0x3F**); here it is 0x27.

Step 3: Test the LCD with Python

Save the following program as **lcd_test.py**:

Code: lcd_test.py

```
from RPLCD.i2c import CharLCD
from time import sleep

# Change 0x27 if i2cdetect shows a different address
lcd = CharLCD('PCF8574', 0x27, cols=16, rows=2)

lcd.write_string("Hello from Pi 400!")
sleep(3)
lcd.clear()
```

Run it:

```
python3 lcd_test.py
```

Optional - display a user-defined message typed at the keyboard:

Code: custom_message.py

```
from RPLCD.i2c import CharLCD
from time import sleep

lcd = CharLCD('PCF8574', 0x27, cols=16, rows=2)

line1 = input("Enter line 1 (max 16 characters): ")
line2 = input("Enter line 2 (max 16 characters): ")

lcd.clear()
lcd.write_string(line1[:16])
lcd.crlf()
lcd.write_string(line2[:16])
sleep(5)
lcd.clear()
```

Output

The LCD displays the message as shown below.



Message displayed on the 16x2 I2C LCD